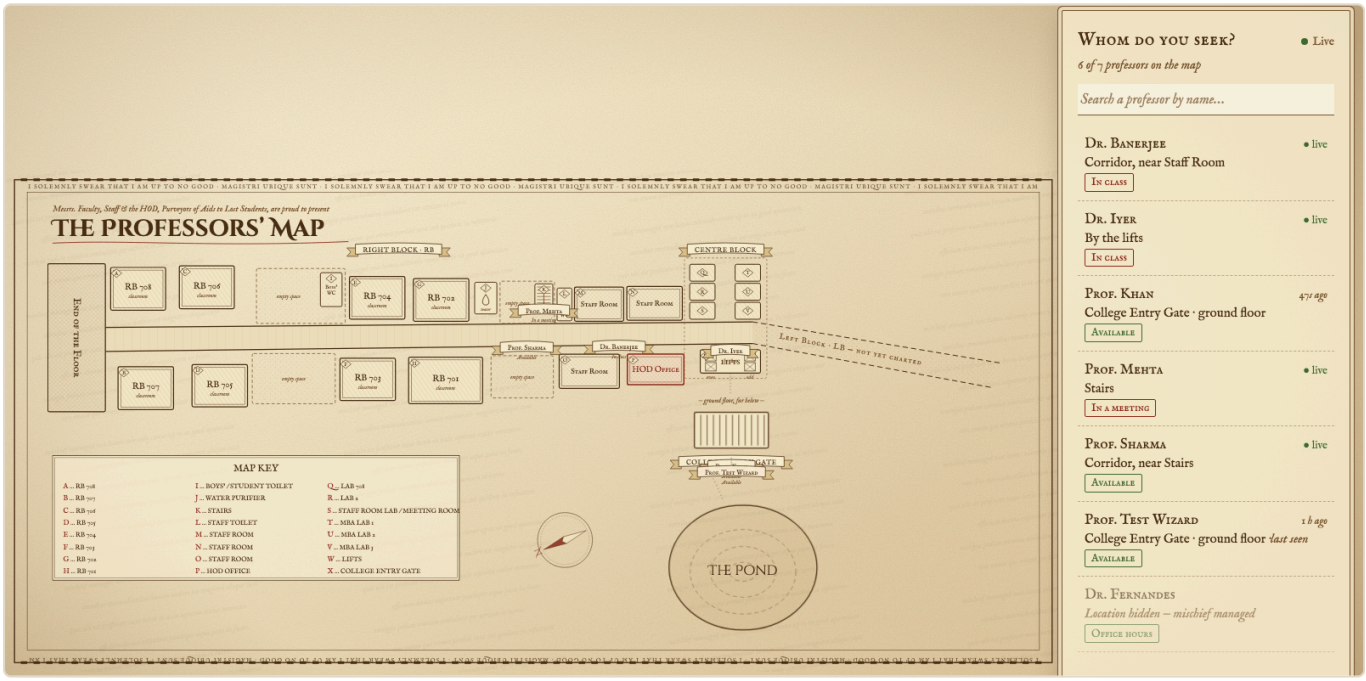
The Professors' Map

Team brief · what it is, how it works, and answers to the questions people will ask · 30 Sep 2026

> **In one line:** a live, Marauder's-Map-style web page that shows students where professors are on the CAC department's 7th floor, fed by the professors' own phones (GPS + QR check-ins).
>
> - **Stack:** plain HTML/CSS/JS in the browser, one small Node.js server using WebSockets (`ws` is the only dependency). No database, no login, no build step.
> - **Opt-in:** a professor is only on the map while they choose to share. Otherwise students see just their name and status.
> - **Hard problem:** phone GPS indoors is noisy and can't tell floors apart. We handle it with a surveyed floor, corridor snapping, and QR codes on doors.
> - **State:** it's a working prototype. It runs locally, has a demo mode, and was tested on phones through a Cloudflare tunnel.



The student view in demo mode (`npm run demo`): the hand-drawn 7th floor, name banners with footprints, and the "Whom do you seek?" list with live status.

1. The pages

URL	Who uses it	What it does
`/`	Students	The map. Professors appear as walking footprints with a name banner. A side list shows each professor's location text, status and "live / last seen", and has a search box.
`/teacher`	Professors	Join with a name, then toggle sharing ("I solemnly swear…" / "Mischief managed"). Pick a status (Available, In class, In a meeting, Office hours, Busy, Away), switch "I'm on the 7th floor", and check in to a room. Location comes from **GPS** or a **simulator** for testing.

`/checkin?room=rb704`	Professors (via QR)	What a door QR code opens. It pins the professor to that room (and floor) for up to 45 minutes.
`/rooms`	Admin	Printable QR codes for every room, lettered A–X to match the map key.
`/survey` + `survey-worker/`	Us (setup only)	The tool we used to walk the floor and record GPS at each landmark. The live copy runs on Cloudflare Workers with a D1 database.

2. Architecture

Professor's phone
`/teacher`, `/checkin`
Browser Geolocation API → smoothed GPS fix sent over WebSocket (on a 2 m move or a 15 s heartbeat)

Node server (`server.js`)
Serves static files + one WebSocket. Holds all professors in an in-memory `Map`. Validates each message, applies the rules, then **broadcasts** the updated professor to every connected client.

Student's browser
`/`
Gets a full snapshot on connect, then one update per change. Turns the position into a map spot and animates footprints (SVG).

Shared brain, used by both server and browsers: `public/config.js` (rooms, surveyed landmarks, thresholds) and `public/geo.js` (GPS → map logic). The same file runs in Node and in the page, so both sides always agree.

WebSocket messages from a phone: `join`, `share`, `status`, `floor`, `loc`, `checkin`, `checkout`, `ping`. **From the server:** `state` (full list on connect) and `update` (one professor). Identity is a random ID saved in the phone's `localStorage`. After a reconnect, the phone re-sends `join` plus its settings, so a server restart heals itself.

3. How a GPS fix becomes a spot on the map

This is the core logic (`GEO.locate` in `geo.js`). Every fix passes these gates in order:

1 Phone reports *lat, lng, ±accuracy*. The phone smooths jitter first and trusts precise fixes more than vague ones.

▼

2 **Accuracy worse than ±35 m?** Ignore it. These are usually cell-tower guesses, and they shouldn't move anyone.

▼

3 **More than 160 m from the 7th-floor lift lobby?** → "Off campus". This also clears the on-floor flag and any room pin.

▼

4 **Not confirmed on floor 7?** → snap to the entry gate or ground-floor lifts if close enough, else show "In the university" (no footprints on the floor plan).

▼

5 **Inside a landmark's radius?** (lifts 5 m, labs 3 m, HOD/staff 2 m, RB 708 6 m…) → snap straight onto that landmark.

▼

6 **Within 16 m of the lifts?** → nearest surveyed centre-block spot ("Centre block, near Lab 2").

▼

7 **Otherwise:** measure how far *along* the right-block corridor the fix is (lifts → RB 708, about 95 m). Map that to a point on the drawn corridor → "Corridor, near RB 704".

Why step 7 works: the right block is basically one long corridor. GPS's sideways error is wider than the corridor itself, so only the distance along it is meaningful. The sketch isn't drawn to scale, so three surveyed "stops" calibrate it: lifts = 0 m, water purifier = 52 m, RB 708 = 95 m. Positions between the stops are linearly interpolated.

4. The floor problem, and QR check-ins

GPS can't tell the 7th floor from the ground floor. So each professor has an **"I'm on the 7th floor"** switch:

- **Turns ON** when they scan any 7th-floor room QR. The lift QR doubles as the "I'm on floor 7" code.
- **Turns OFF** when they leave campus (>160 m), after **4 hours**, when they scan the entry-gate QR, or when they flip it manually.

QR check-in is the most accurate signal we have. It pins the professor to the room for **45 minutes**, and their raw position moves to the room too, so the dot doesn't jump. A confident GPS fix (±20 m or better) that lands far from the room (roughly 16 m on the sketch) cancels the pin early. So walking away overrides a stale scan.

5. The floor survey (where the numbers came from)

We walked the floor with `/survey`. At each landmark we stood still and recorded about 20 s of GPS. Every recording goes to a Cloudflare D1 database (`floor7-survey-db`). It queues on the phone if the network drops and retries without duplicates, because IDs are generated on the phone. Important spots were recorded 2–3 times. The final value is the **median** of the starred ("this is the real one") recordings. Deletes are soft, so no reading is ever lost. The current export has 19 recordings (lifts, labs, meeting room, staff rooms, HOD, stairs, purifier, RB 706/708, gate, etc.). The results are hard-coded in `config.js`.

6. Key numbers (all in `public/config.js`)

Setting	Value	Meaning

`maxAcc`	35 m	Fixes vaguer than this are ignored
`campusRadius`	160 m	Beyond this from the lifts = off campus
`centre.radius`	16 m	Around the lifts = centre block
`floorHoldHours`	4 h	“On floor 7” expires unless refreshed
`roomPinMinutes`	45 min	How long a QR check-in holds
Phone send rule	2 m / 15 s	Send on a 2 m move, or a heartbeat every 15 s
Map “live” / “stale”	30 s / 5 min	Badge shows “live” under 30 s. The banner fades after 5 min of silence.

7. Known limits (be upfront about these)

- **Left Block isn’t surveyed.** It’s drawn (dashed, “not yet charted”), but nobody is ever placed there.
- **No background tracking.** Browsers stop sending location when the phone is locked. The page asks for a screen wake lock. Real background tracking needs a native app.
- **Everything is in memory.** A server restart clears the state. Phones rejoin automatically, but positions only reappear with the next fix.
- **No authentication.** Anyone can join with any name, and anyone who knows a `/checkin?room=…` URL can check in there.
- **GPS needs HTTPS** on phones. For testing we use `npx cloudflared tunnel --url http://localhost:3000`.
- **HOD landmark is weak.** Its survey came back at ±63 m accuracy, so it only snaps within 2 m.

> **Privacy caveat worth knowing before someone asks:** while a professor is sharing, the server sends their **raw latitude/longitude** to every map viewer. The student page turns those into a map spot in the browser. Anyone opening dev tools could read the coordinates. The fix is simple: run `GEO.locate` on the server and broadcast only the map spot and label. It’s on the to-do list.

8. Cross-questions, with answers

Concept & choices

Why not Wi-Fi positioning, BLE beacons or a proper indoor-positioning system?

Those need hardware or access to the college network. GPS + QR needs nothing beyond professors' phones and printed paper. It gets us room-level accuracy where it matters (QR) and corridor-level accuracy the rest of the time. Beacons are a natural upgrade later, and the map logic wouldn't change.

How accurate is it, honestly?

A QR check-in is exact to the room. GPS inside the building is typically ±5–20 m, so we only claim things like "Corridor, near RB 704" or "Centre block, near Lab 2". We never claim a specific room from GPS alone unless the fix sits right on a surveyed landmark. Fixes worse than ±35 m are thrown away.

How do you know which floor someone is on?

We don't, from GPS. We use the explicit switch, QR scans (a 7th-floor room QR turns it on, the gate QR turns it off), a 4-hour expiry, and an automatic reset when they leave campus. Off-floor professors show as "In the university" or at the gate, never on the 7th-floor plan.

Why squash positions onto the corridor instead of plotting raw GPS?

Raw GPS would draw people inside walls and rooms they aren't in. The right block is one corridor, and GPS's sideways error is bigger than the corridor's width. The only trustworthy number is how far along it you are, so we project onto that line.

Why no framework or database?

It's about 2,000 lines total. A framework would add build tooling and weight for no gain. A database isn't needed because live location is only useful right now, and we deliberately keep no history.

Why the Harry Potter theme?

It makes the product fun for students and gives professors a memorable on/off gesture ("I solemnly swear…" / "Mischief managed"). The theme is purely visual, and the logic is independent of it.

Privacy & trust

Is this tracking professors?

Only with consent, and only while they choose. Sharing is off by default. One tap hides them, and students then see only the name and status. Nothing is stored: no history, no logs of positions, and a restart wipes everything. Location also only means anything on campus. Off campus it just says "Off campus".

Can students see exact coordinates?

The UI doesn't show them, but today the raw coordinates do reach the browser (see the caveat in section 7). Be honest here and say it's a known, small fix: compute the spot on the server and send only that.

Can someone fake being a professor or fake a location?

Yes, right now. There's no login, and the QR URLs are guessable. That's acceptable for a prototype. For a real rollout: college SSO for professors, and signed or rotating QR codes.

Reliability

What if a professor forgets to turn sharing off?

If they leave campus, they automatically show as "Off campus" and their floor/room flags clear. If their phone goes quiet, the "live" badge turns into "last seen Xm ago", and after 5 minutes their banner fades on the map.

What happens when the phone is locked or the tab is in the background?

The browser pauses GPS. We request a screen wake lock and send a heartbeat every 15 s while active. True background tracking needs a native app, which is the biggest limitation of a web-only approach.

What if the server restarts or the network drops?

Clients reconnect with backoff (0.5 s up to 8 s) and re-send their identity and settings. Non-location messages are queued while offline. State lives in memory, so positions reappear on the next GPS fix.

Battery drain?

High-accuracy GPS plus a screen kept awake is costly. Mitigations: we only send on a 2 m move or a 15 s heartbeat, and QR check-in lets a professor pin themselves without keeping GPS on. A native app could use the OS's cheaper geofencing.

Scale, cost, next steps

Will it scale?

One Node process broadcasts each change to every connected viewer. That's comfortably fine for a department (dozens of professors, hundreds of students). For a whole campus we'd throttle broadcasts, only send what changed, and possibly shard by floor.

What does it cost to run?

Essentially nothing. Any small Node host works, plus a domain with HTTPS. The survey Worker and D1 fit in Cloudflare's free tier.

How do we add another floor or block?

Survey the landmarks with `/survey`, add rooms, landmarks and corridor stops to `config.js`, and print new QR codes from `/rooms`. The Left Block on floor 7 is the obvious next one.

What's next?

(1) Server-side location so raw coordinates never leave the server. (2) Login for professors and signed QR codes. (3) Survey the Left Block. (4) Persist state across restarts. (5) Possibly a native app or BLE beacons for background and room-level accuracy.

9. Running it

```
npm install
npm run demo      # map + 6 wandering demo professors
npm start         # real professors only
npx cloudflared tunnel --url http://localhost:3000   # HTTPS, for GPS on phones
```

Open `http://localhost:3000/` for the map and `/teacher` for the professor side. The simulator mode on `/teacher` works over plain http, with no phone needed.

Where things live

File	What's in it
`server.js`	Static server, WebSocket handling, professor state, check-in/floor rules, demo bots
`public/config.js`	Rooms, surveyed landmarks, thresholds: the one place to tune
`public/geo.js`	GPS ↔ metres ↔ map maths and `locate()` (shared by server and browser)
`public/map.js`, `illustrate.js`	The student map: SVG parchment, footprints, list
`public/teacher.js`, `client.js`	Professor page (GPS, simulator, sharing), identity + reconnecting socket
`survey-worker/`	Cloudflare Worker + D1 schema for the floor survey

Prepared from the codebase as of 30 Sep 2026. All thresholds are quoted from `public/config.js` and may be tuned later.